

Documentação Técnica — Registo de Decisões Arquiteturais e Tecnológicas

Architecture Decision Record (ADR)
Total de decisões registadas: 12 | Atualizado em: 03/07/2026

Este documento regista as decisões arquiteturais e tecnológicas efetivamente presentes no sistema SkyLearn, tal como implementadas no código do workspace. Cada decisão é descrita no formato Architecture Decision Record (ADR): contexto, decisão tomada, alternativas não adotadas e consequências. O objetivo é servir de referência técnica para a equipa e como base de apoio ao diagrama de arquitetura, sem depender de visões aspiracionais do sistema.

Índice de decisões

ID	Título	Categoria	
ADR-01	Arquitetura geral do backend: monólito modular em vez de microserviços	Arquitetura	Aceite
ADR-02	Base de dados relacional: MySQL com schema gerido pelo Hibernate	Dados	Aceite
ADR-03	Frontend Web: SPA em React + Vite com camada de API dedicada	Frontend	Aceite
ADR-04	Aplicação Mobile: React Native + Expo Router, cliente único adaptado por papel	Frontend	Aceite
ADR-05	Autenticação e autorização: JWT Bearer com blacklist para logout	Segurança	Aceite
ADR-06	Modelo de papéis (RBAC) com seis perfis de acesso	Segurança	Aceite
ADR-07	Fluxo editorial de conteúdo como máquina de estados explícita	Domínio / Negócio	Aceite
ADR-08	Editor de conteúdo rich-text baseado em HTML editável no browser	Frontend / Domínio	Aceite
ADR-09	Armazenamento de media: filesystem local servido pelo próprio backend	Infraestrutura	Aceite
ADR-10	Documentação de API com springdoc OpenAPI (Swagger)	Qualidade / Documentação	Aceite
ADR-11	Notificações por email via SMTP (Spring Mail)	Integrações	Aceite
ADR-12	Ausência deliberada de cache distribuído e filas de mensagens	Arquitetura	Aceite

Registo detalhado das decisões

ADR-01 — Arquitetura geral do backend: monólito modular em vez de microserviços

Categoria	Arquitetura
Estado	Aceite / Implementado
Contexto	O sistema precisa de suportar autenticação, conteúdos, quizzes, fórum, perfis, notificações e upload de media, para dois clientes (web e mobile).
Decisão	Adotar um backend monolítico em Spring Boot (Java 17), com separação interna por camadas (Controller, Service, Repository, DTO, Model/Entity, Config) e recursos da API versionados em /v1, expostos sob o contexto /api.
Alternativas consideradas	Arquitetura de microserviços e filas de mensagens entre serviços — não implementadas no código atual.

Consequências

- Deployment e desenvolvimento mais simples numa fase de MVP, com um único processo a gerir.
- Menor complexidade operacional (sem orquestração de vários serviços).
- Escalabilidade independente por módulo (ex.: escalar só o upload de media) fica limitada enquanto se mantiver o monólito.

ADR-02 — Base de dados relacional: MySQL com schema gerido pelo Hibernate

Categoria	Dados
Estado	Aceite / Implementado
Contexto	O sistema precisa de persistir utilizadores, roles, conteúdos, quizzes, tentativas, fórum, notificações e estatísticas de forma relacional.
Decisão	Usar MySQL como base de dados, acedida via Spring Data JPA/Hibernate, com a opção ddl-auto em modo update para gerar e atualizar o schema automaticamente a partir das entidades.
Alternativas consideradas	PostgreSQL e uso de Flyway para migrations versionadas — nenhum dos dois está presente no código.

Consequências

- Redução do esforço inicial: não é preciso escrever migrations manuais.
- Risco de deriva de esquema entre ambientes, por não existir um histórico de migrations versionado e reprodutível.
- Recomenda-se reavaliar a introdução de Flyway antes de um ambiente de produção com múltiplos deployments.

ADR-03 — Frontend Web: SPA em React + Vite com camada de API dedicada

Categoria	Frontend
Estado	Aceite / Implementado
Contexto	É necessária uma interface web para estudantes e para o backoffice editorial (escritores, revisores, aprovadores e administração).
Decisão	Construir o frontend web como uma SPA em React, com Vite como bundler/dev server, React Router para navegação, um AuthProvider central para autenticação e permissões, e uma camada backendApi para comunicação com o backend. Existe um fallback para dados mock quando a API não responde.
Alternativas consideradas	Não há evidência no código de outras stacks (ex.: Next.js, Angular) terem sido avaliadas ou usadas.

Consequências

- Centralizar a autenticação num único provider facilita aplicar controlo de acesso por papel na interface.
- O fallback para mock data aumenta a resiliência percebida pelo utilizador em caso de indisponibilidade do backend, mas pode mascarar falhas reais se não for sinalizado claramente.

ADR-04 — Aplicação Mobile: React Native + Expo Router, cliente único adaptado por papel

Categoria	Frontend
Estado	Aceite / Implementado
Contexto	É necessário um cliente mobile que sirva sobretudo estudantes, mas que também contemple os restantes papéis (escritor, revisor, aprovador, admin, superadmin).
Decisão	Implementar a aplicação mobile com React Native e Expo Router (navegação por ficheiros), com um layout raiz que injeta tema e providers da aplicação. É um único cliente que adapta rotas e UI ao papel autenticado, em vez de apps separadas por perfil.
Alternativas consideradas	Não há evidência de terem sido avaliadas apps nativas separadas por plataforma ou por perfil de utilizador.

Consequências

- Reutilização de código e de fluxos de autenticação entre perfis.
- Limitação atual conhecida: não existe estratégia offline-first completa, sincronização em fila, nem integração FCM fechada. Deve ser tratado como lacuna a decidir em trabalho futuro, não como funcionalidade implementada.

ADR-05 — Autenticação e autorização: JWT Bearer com blacklist para logout

Categoria	Segurança
Estado	Aceite / Implementado
Contexto	Os dois clientes (web e mobile) precisam de autenticar utilizadores e aplicar autorização baseada em papel nos endpoints da API.
Decisão	Usar JWT no header Authorization (Bearer). O JwtAuthFilter valida o token em cada pedido e popula o contexto de segurança; o SecurityConfig define quais rotas são públicas, quais exigem autenticação e quais exigem um papel específico. O logout é implementado por blacklist de tokens.
Alternativas consideradas	Refresh token guardado em cookie HttpOnly — não implementado no fluxo atual.

Consequências

- Implementação mais simples de autenticação stateless entre clientes e API.
- A ausência de refresh token HttpOnly cookie implica maior exposição do token do lado do cliente; a blacklist de tokens é necessária para invalidar sessões no logout, o que introduz estado no servidor.

ADR-06 — Modelo de papéis (RBAC) com seis perfis de acesso

Categoria	Segurança
Estado	Aceite / Implementado
Contexto	Diferentes utilizadores precisam de permissões distintas: consumir conteúdo, criar conteúdo, rever, aprovar, administrar ou ter controlo total.
Decisão	Definir seis papéis aplicados tanto na API como na lógica de UI: ESTUDANTE, ESCRITOR, REVISOR, APROVADOR, ADMIN e SUPERADMIN. As regras de autorização por recurso (conteúdo, quiz, fórum, ficheiros, admin) são expressas em função destes papéis.
Alternativas consideradas	Não há evidência de um modelo mais granular (permissões por ação, ACLs) ter sido considerado.

Consequências

- O fluxo editorial de conteúdo depende diretamente deste modelo de papéis para separar quem cria, revê e aprova.
- Um modelo baseado em papéis fixos é simples de raciocinar, mas menos flexível do que permissões granulares caso surjam exceções por utilizador.

ADR-07 — Fluxo editorial de conteúdo como máquina de estados explícita

Categoria	Domínio / Negócio
Estado	Aceite / Implementado
Contexto	O conteúdo publicado na plataforma precisa de passar por revisão e aprovação antes de ficar visível a estudantes.
Decisão	Codificar no backend os estados DRAFT, UNDER_REVIEW, APPROVED, PUBLISHED e REJECTED para cada conteúdo, com transições geridas pelos papéis ESCRITOR (cria/submete), REVISOR (avalia), APROVADOR (publica ou rejeita) e ADMIN/SUPERADMIN (podem intervir com permissões ampliadas).
Alternativas consideradas	Não há evidência de um motor de workflow genérico (BPMN, state machine externa) ter sido usado; os estados são geridos diretamente na lógica de negócio.

Consequências

- Rastreabilidade clara do ciclo de vida de cada conteúdo.
- A lógica de transição de estados fica concentrada na camada de serviço, o que exige disciplina para evitar transições inválidas à medida que o sistema cresce.

ADR-08 — Editor de conteúdo rich-text baseado em HTML editável no browser

Categoria	Frontend / Domínio
Estado	Aceite / Implementado
Contexto	Os autores de conteúdo precisam de produzir texto formatado (negrito, listas, títulos, ligações, media incorporada) sem depender de um serviço externo de authoring.
Decisão	Implementar um editor rich-text inline na página de publicação, baseado em comandos de edição nativos do browser sobre um elemento editável, que persiste o conteúdo como HTML (incluindo formatação, listas, citações, alinhamento, cabeçalhos H1–H3, ligações e media).
Alternativas consideradas	Não há evidência de uso de uma biblioteca de editor de terceiros (ex.: TipTap, Slate, Quill) nem de um formato estruturado alternativo (ex.: Markdown, JSON de blocos).

Consequências

- Permite formatação rica sem dependências externas pesadas.
- Como o conteúdo é HTML gerado pelo utilizador, deve ser tratado como ponto de atenção de segurança: recomenda-se garantir sanitização do HTML antes de o apresentar a outros utilizadores, para mitigar risco de XSS.

ADR-09 — Armazenamento de media: filesystem local servido pelo próprio backend

Categoria	Infraestrutura
Estado	Aceite / Implementado
Contexto	O sistema precisa de aceitar e servir ficheiros de vídeo, áudio e imagem associados aos conteúdos.
Decisão	Implementar o upload de media no próprio backend: o frontend envia multipart/form-data, o FileStorageService valida mime type, extensão e tamanho, e o ficheiro é guardado localmente em uploads/videos, uploads/audios ou uploads/images. O backend devolve e serve a URL de download.
Alternativas consideradas	Object storage (ex.: S3-like) e CDN para distribuição de media — nenhum dos dois está presente no código.

Consequências

- Simplifica a implementação para o estado atual de MVP, sem dependências de infraestrutura externa.
- O backend acumula a responsabilidade de API e de servidor de ficheiros no mesmo processo, o que limita escalabilidade e redundância de media à medida que o volume crescer.

ADR-10 — Documentação de API com springdoc OpenAPI (Swagger)

Categoria	Qualidade / Documentação
Estado	Aceite / Implementado
Contexto	Os clientes web e mobile, e potenciais integradores, precisam de um contrato claro dos endpoints disponíveis.
Decisão	Integrar springdoc OpenAPI no backend Spring Boot para gerar documentação Swagger da API automaticamente a partir do código.
Alternativas consideradas	Documentação de API mantida manualmente fora do código — não é a abordagem usada.

Consequências

- O contrato da API fica sempre sincronizado com o código, reduzindo divergência entre documentação e implementação.

ADR-11 — Notificações por email via SMTP (Spring Mail)

Categoria	Integrações
Estado	Aceite / Implementado
Contexto	O sistema precisa de comunicar com utilizadores fora da aplicação, nomeadamente para recuperação de password e notificações.
Decisão	Integrar Spring Mail com um servidor SMTP como único serviço externo de comunicação, usado para recuperação de password e notificações por email.
Alternativas consideradas	Filas de mensagens para envio assíncrono de notificações — não implementadas.

Consequências

- Simplicidade de integração para o volume atual.
- Sem uma fila de mensagens, o envio de email fica dependente da disponibilidade síncrona do servidor SMTP no momento do pedido, sem mecanismo de retry assíncrono desacoplado.

ADR-12 — Ausência deliberada de cache distribuído e filas de mensagens

Categoria	Arquitetura
Estado	Aceite / Implementado
Contexto	Componentes como Redis (cache/sessões) e filas de mensagens são comuns em plataformas com maior escala ou processamento assíncrono.
Decisão	Não incluir, no estado atual do sistema, Redis, filas de mensagens, CDN ou object storage. O sistema funciona apenas com Spring Boot, MySQL, filesystem local e SMTP.
Alternativas consideradas	Redis para cache/sessões e um message broker para processamento assíncrono — nenhum dos dois está implementado.

Consequências

- Superfície de infraestrutura reduzida, mais fácil de operar e depurar na fase atual do projeto.
- Sem cache distribuído nem processamento assíncrono desacoplado, operações mais pesadas (ex.: envio de notificações em massa, cálculos de ranking) correm de forma síncrona e devem ser revistas se a carga aumentar.